

设计模式组合考题试卷二

第 6 题：订单处理系统

题目

某电商平台需要实现一个订单处理系统，所有订单的处理流程都是固定的：

- 1. 验证订单信息（检查商品库存、用户地址是否有效）
- 2. 计算订单总金额
- 3. 执行支付操作
- 4. 生成物流单并发货

系统要求：

- 1. 不同类型的订单（普通订单、预售订单、团购订单）的支付方式可以不同
- 2. 普通订单支持微信、支付宝、银行卡支付
- 3. 预售订单只支持支付宝支付
- 4. 团购订单只支持微信支付

需要实现的类

- 1. **Order****（订单类）**
 - 功能：表示一个订单，包含订单 ID、金额、支付方式等属性
- 2. **PaymentMethod****接口及各类支付方式实现类**
 - 功能：定义支付的统一方法，并实现不同支付方式的逻辑
 - 包含方法：void pay(double amount)
- 3. **PaymentMethodProvider****（支付方式提供类）**
 - 功能：根据支付方式名称提供对应的支付方式实现
 - 包含方法：[支付方式类类型] getPaymentMethod(String methodName)
- 4. **OrderProcessor****抽象类及各类订单处理器实现类**
 - 功能：定义订单处理的固定流程，并实现不同类型订单的支付逻辑
 - 包含方法：
 - final void processOrder(Order order)：订单处理的固定流程（验证→计算→支付→发货）
 - abstract void doPayment(Order order)：抽象支付方法，由子类实现

系统要求

- 1. 订单处理的整体流程不能改变，只能修改支付步骤的实现
- 2. 不同类型的订单使用不同的支付方式
- 3. 未来可以方便地新增订单类型和支付方式

第 7 题：第三方支付整合系统

题目

某系统需要整合多个第三方支付接口：

- 支付宝接口：提供pay(String orderId, double amount)方法
- 微信支付接口：提供wxPay(String outTradeNo, double totalFee)方法
- 银联支付接口：提供unionPay(String merOrderId, double txnAmt)方法

需要实现的类

1. **AlipayService****/WechatPayService/UnionPayService（第三方支付服务类）**

- 功能：模拟第三方支付接口，包含各自的支付方法
- 注意：这三个类的方法名和参数不同，且不能修改

2. 各类支付接口的转换类

- 功能：将第三方支付的不同接口转换成统一的调用方式
- 每个类包含方法：void unifiedPay(String orderId, double amount)

3. **PaymentAdapterProvider****（支付转换类提供类）**

- 功能：根据支付方式提供对应的转换类
- 包含方法：[转换类类型] getPaymentAdapter(String paymentType)

4. **UnifiedPayment****（统一支付类）**

- 功能：对外提供统一的支付入口
- 包含方法：void pay(String paymentType, String orderId, double amount)

系统要求

1. 客户端通过UnifiedPayment类调用所有第三方支付，无需知道第三方接口的具体实现
2. 未来可以方便地新增第三方支付接口，无需修改现有客户端代码

第 8 题：咖啡店订单系统

题目

某咖啡店需要实现一个订单系统，提供多种咖啡和配料：

- 基础咖啡：美式咖啡（20 元）、拿铁咖啡（25 元）、卡布奇诺（28 元）
- 配料：牛奶（5 元）、糖（2 元）、奶泡（3 元）、巧克力酱（6 元）

需要实现的类

1. **Coffee****接口及各类基础咖啡实现类**
 - 功能：定义咖啡的统一方法，并实现不同基础咖啡的价格和描述
 - 包含方法：double getPrice()、String getDescription()
2. **CoffeeProvider****（咖啡提供类）**
 - 功能：根据咖啡类型提供对应的基础咖啡对象
 - 包含方法：Coffee getCoffee(String type)
3. 各类配料的包装类
 - 功能：为咖啡添加配料，并动态计算价格和更新描述
 - 每个类包含方法：double getPrice()、String getDescription()
 - 构造函数：接收一个Coffee对象作为参数

系统要求

1. 客户可以选择任意一种基础咖啡，并添加任意多种配料
2. 系统可以动态计算订单的总价格和完整描述
3. 未来可以方便地新增咖啡种类和配料

第 9 题：文件系统目录树构建

题目

某文件管理系统需要实现一个目录树构建功能：

1. 文件系统由文件和目录组成，目录可以包含文件和子目录
2. 可以通过指定的路径构建完整的目录树
3. 支持遍历目录树，打印所有文件和目录的名称

需要实现的类

1. **FileSystemNode****抽象类**
 - 功能：文件和目录的共同父类，定义统一的操作接口

- 包含方法：

- void add(FileSystemNode node): 添加子节点
- void remove(FileSystemNode node): 删除子节点
- void display(int depth): 显示节点信息 (depth 表示缩进深度)

2. **File**** (文件类) **

- 功能：表示文件系统中的文件
- 继承自FileSystemNode，实现对应的方法

3. **Directory**** (目录类) **

- 功能：表示文件系统中的目录，可以包含文件和子目录
- 继承自FileSystemNode，实现对应的方法

4. **DirectoryTreeBuilder**** (目录树构建类) **

- 功能：一步步构建完整的目录树
- 包含方法：
 - DirectoryTreeBuilder(String rootName): 构造函数，指定根目录名称
 - DirectoryTreeBuilder addFile(String path, String fileName): 在指定路径下添加文件
 - DirectoryTreeBuilder addDirectory(String path, String dirName): 在指定路径下添加子目录
 - Directory build(): 返回构建好的根目录

系统要求

1. 目录可以包含文件和子目录，形成树状结构
2. 可以通过DirectoryTreeBuilder方便地构建复杂的目录树
3. 支持遍历目录树，以缩进形式显示所有文件和目录

第 10 题：电梯控制系统

题目

某电梯控制系统需要实现电梯的基本功能：

- 1. 电梯有四种状态：开门状态、关门状态、运行状态、停止状态
- 2. 不同状态下电梯可以执行的操作不同：
 - 开门状态：只能执行关门操作
 - 关门状态：可以执行开门、上行、下行操作
 - 运行状态：不能执行任何操作，直到到达目标楼层
 - 停止状态：可以执行开门、上行、下行操作
- 3. 电梯到达目标楼层后会自动切换到停止状态

需要实现的类

- 1. **RunStrategy****接口及各类运行策略实现类**
 - 功能：定义电梯运行的统一方法，并实现不同的运行策略（如快速运行、节能运行）
 - 包含方法：void run(int fromFloor, int toFloor)
- 2. **ElevatorState****接口及各类电梯状态实现类**
 - 功能：定义电梯在不同状态下的操作方法，并实现各状态的具体逻辑
 - 包含方法：void openDoor()、void closeDoor()、void goUp(int floor)、void goDown(int floor)
- 3. **Elevator****（电梯类）**
 - 功能：表示电梯，管理电梯状态、当前楼层和运行策略
 - 包含方法：
 - void openDoor()、void closeDoor()、void goUp(int floor)、void goDown(int floor)：委托给当前状态类执行
 - void setState([状态类类型] state)：切换电梯状态
 - getter 和 setter 方法：getCurrentFloor()、setCurrentFloor()、getRunStrategy()、setRunStrategy()

系统要求

- 1. 电梯的行为由其当前状态决定，不同状态下有不同的操作限制
- 2. 电梯到达目标楼层后会自动切换状态
- 3. 可以灵活切换电梯的运行策略（快速 / 节能）
- 4. 未来可以方便地新增电梯状态和运行策略

